.secrets/streamlit.toml

[mysql]

host = "localhost"

port = 3306

user = "jagdevsinghdosanjh"

password = "Kawaljit@1973"

database = "school_mgmt"

auth/auth_manager.py

```
import bcrypt
import streamlit as st
from .db_manager import get_user_by_username, get_roles_for_user


def hash_password(plain_password: str) -> str:
    return bcrypt.hashpw(plain_password.encode(), bcrypt.gensalt()).decode()


def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode(), hashed_password.encode())


def login_form():
    st.subheader("Login")
    username = st.text_input("Username")
    password = st.text_input("Password", type="password")
    if st.button("Login"):
        user = get_user_by_username(username)
        if not user or not user["is_active"]:
            st.error("Invalid username or inactive account.")
            return
        if not verify_password(password, user["password_hash"]):
            st.error("Incorrect password.")
            return

        roles = get_roles_for_user(user["id"])
        if not roles:
            st.error("No roles assigned. Contact admin.")
            return

        st.session_state["authenticated"] = True
        st.session_state["user"] = {
```

```
            "id": user["id"],
            "username": user["username"],
            "full_name": user["full_name"],
            "email": user["email"],
            "roles": roles,
        }
        st.experimental_rerun()


def logout():
    for key in ["authenticated", "user"]:
        st.session_state.pop(key, None)
    st.experimental_rerun()


def is_authenticated() -> bool:
    return st.session_state.get("authenticated", False)


def has_role(*required_roles) -> bool:
    if not is_authenticated():
        return False
    user_roles = st.session_state["user"]["roles"]
    return any(r in user_roles for r in required_roles)
```

auth/db_manager.py

```
import mysql.connector
from typing import Optional, Dict, List
import streamlit as st


def get_connection():
    return mysql.connector.connect(
        host=st.secrets["mysql"]["host"],
        port=st.secrets["mysql"]["port"],
        user=st.secrets["mysql"]["user"],
        password=st.secrets["mysql"]["password"],
        database=st.secrets["mysql"]["database"],
    )


def get_user_by_username(username: str) -> Optional[Dict]:
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT id, username, password_hash, full_name, email, is_active
```

```
        FROM users
        WHERE username = %s
    """,
        (username,),
    )
    row = cur.fetchone()
    cur.close()
    conn.close()
    if row:
        return {
            "id": row[0],
            "username": row[1],
            "password_hash": row[2],
            "full_name": row[3],
            "email": row[4],
            "is_active": bool(row[5]),
        }
    return None


def get_roles_for_user(user_id: int) -> List[str]:
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT r.name
        FROM roles r
        JOIN user_roles ur ON r.id = ur.role_id
        WHERE ur.user_id = %s
    """,
        (user_id,),
    )
    roles = [r[0] for r in cur.fetchall()]
    cur.close()
    conn.close()
    return roles
```

db/attendance.py

```
from typing import Dict, List
from datetime import date
from .base import get_connection


def save_attendance(
```

```
    class_name: str, att_date: date, teacher_id: int, status_map: Dict[int,
str]
):
    """
    status_map: {student_id: 'present' or 'absent'}
    """
    conn = get_connection()
    cur = conn.cursor()

    for student_id, status in status_map.items():
        cur.execute(
            """
            INSERT INTO attendance (student_id, class, date, status,
teacher_id)
            VALUES (%s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE status = VALUES(status), teacher_id =
VALUES(teacher_id)
        """,
            (student_id, class_name, att_date, status, teacher_id),
        )

    conn.commit()
    cur.close()
    conn.close()


def get_attendance_for_student(student_id: int):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT date, status
        FROM attendance
        WHERE student_id = %s
        ORDER BY date DESC
    """,
        (student_id,),
    )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return rows


def get_attendance_summary(student_id: int):
    conn = get_connection()
    cur = conn.cursor()
```

```
    cur.execute(
        """
        SELECT
            SUM(CASE WHEN status = 'present' THEN 1 ELSE 0 END) AS
present_days,
            SUM(CASE WHEN status = 'absent' THEN 1 ELSE 0 END) AS absent_days
        FROM attendance
        WHERE student_id = %s
    """,
        (student_id,),
    )
    row = cur.fetchone()
    cur.close()
    conn.close()
    present = row[0] or 0
    absent = row[1] or 0
    total = present + absent
    percent = (present / total * 100) if total > 0 else 0
    return {
        "present": present,
        "absent": absent,
        "total": total,
        "percent": round(percent, 2),
    }


def get_attendance_for_class_date(class_name: str, att_date: date):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT s.full_name, s.roll_no, a.status
        FROM attendance a
        JOIN students s ON a.student_id = s.id
        WHERE a.class = %s AND a.date = %s
        ORDER BY s.roll_no
    """,
        (class_name, att_date),
    )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return rows
```

db/base.py

```
import mysql.connector
import streamlit as st
```

```python
def get_connection():
    return mysql.connector.connect(
        host=st.secrets["mysql"]["host"],
        port=st.secrets["mysql"]["port"],
        user=st.secrets["mysql"]["user"],
        password=st.secrets["mysql"]["password"],
        database=st.secrets["mysql"]["database"],
    )
```

db/fees.py

```python
import mysql.connector
import streamlit as st
from datetime import date


def get_connection():
    return mysql.connector.connect(
        host=st.secrets["mysql"]["host"],
        port=st.secrets["mysql"]["port"],
        user=st.secrets["mysql"]["user"],
        password=st.secrets["mysql"]["password"],
        database=st.secrets["mysql"]["database"],
    )


# ----------------------------------------------------------
# Initialize fees record for a new student
# ----------------------------------------------------------
def initialize_fee_record(student_id: int, total_fees: int):
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        """
        INSERT INTO fees (student_id, total_fees, paid, last_payment_date)
        VALUES (%s, %s, 0, NULL)
    """,
        (student_id, total_fees),
    )

    conn.commit()
    cur.close()
    conn.close()


# ----------------------------------------------------------
```

```python
# Record a fee payment
# ---------------------------------------------------------
def record_fee_payment(student_id: int, amount: int, mode: str):
    conn = get_connection()
    cur = conn.cursor()

    # Insert into fee_payments table
    cur.execute(
        """
        INSERT INTO fee_payments (student_id, amount, mode)
        VALUES (%s, %s, %s)
    """,
        (student_id, amount, mode),
    )

    # Update fees table
    cur.execute(
        """
        UPDATE fees
        SET paid = paid + %s,
            last_payment_date = CURRENT_TIMESTAMP
        WHERE student_id = %s
    """,
        (amount, student_id),
    )

    conn.commit()
    cur.close()
    conn.close()


# ---------------------------------------------------------
# Get fee status for a student
# ---------------------------------------------------------
def get_fee_status(student_id: int):
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        """
        SELECT total_fees, paid, due, last_payment_date
        FROM fees
        WHERE student_id = %s
    """,
        (student_id,),
    )
```

```python
    row = cur.fetchone()
    cur.close()
    conn.close()

    if row:
        return {
            "total_fees": row[0],
            "paid": row[1],
            "due": row[2],
            "last_payment_date": row[3],
        }
    return None


# ---------------------------------------------------------
# Get payment history
# ---------------------------------------------------------
def get_payment_history(student_id: int):
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        """
        SELECT amount, mode, payment_date
        FROM fee_payments
        WHERE student_id = %s
        ORDER BY payment_date DESC
    """,
        (student_id,),
    )

    rows = cur.fetchall()
    cur.close()
    conn.close()

    return rows
```

db/marks.py

```python
from typing import List, Dict
from .base import get_connection


def save_marks(entries: List[Dict]):
    """
    entries: list of dicts like:
    {
```

```
        "student_id": int,
        "subject": str,
        "exam": str,
        "marks_obtained": int,
        "max_marks": int,
        "teacher_id": int
    }
    """
    conn = get_connection()
    cur = conn.cursor()

    for e in entries:
        cur.execute(
            """
            INSERT INTO marks (student_id, subject, exam, marks_obtained,
max_marks, teacher_id)
            VALUES (%s, %s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE
                marks_obtained = VALUES(marks_obtained),
                max_marks = VALUES(max_marks),
                teacher_id = VALUES(teacher_id)
        """,
            (
                e["student_id"],
                e["subject"],
                e["exam"],
                e["marks_obtained"],
                e.get("max_marks", 100),
                e["teacher_id"],
            ),
        )

    conn.commit()
    cur.close()
    conn.close()


def get_marks_for_student(student_id: int, exam: str):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT subject, marks_obtained, max_marks
        FROM marks
        WHERE student_id = %s AND exam = %s
        ORDER BY subject
    """,
```

```
        (student_id, exam),
    )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return rows


def get_marks_for_class_exam(class_name: str, exam: str, subject: str):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT s.full_name, s.roll_no, m.marks_obtained, m.max_marks
        FROM marks m
        JOIN students s ON m.student_id = s.id
        WHERE s.class = %s AND m.exam = %s AND m.subject = %s
        ORDER BY s.roll_no
    """,
        (class_name, exam, subject),
    )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return rows
```

db/students.py

```
from typing import Optional, Dict, List
from .base import get_connection


def register_student(
    full_name: str, roll_no: str, cls: str, section: str = None, parent_id:
int = None
) -> int:
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        INSERT INTO students (full_name, roll_no, class, section, parent_id)
        VALUES (%s, %s, %s, %s, %s)
    """,
        (full_name, roll_no, cls, section, parent_id),
    )
    conn.commit()
    student_id = cur.lastrowid
```

```
    cur.close()
    conn.close()
    return student_id


def get_student_by_roll(roll_no: str) -> Optional[Dict]:
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT id, full_name, roll_no, class, section, parent_id
        FROM students
        WHERE roll_no = %s
    """,
        (roll_no,),
    )
    row = cur.fetchone()
    cur.close()
    conn.close()
    if row:
        return {
            "id": row[0],
            "full_name": row[1],
            "roll_no": row[2],
            "class": row[3],
            "section": row[4],
            "parent_id": row[5],
        }
    return None


def get_student_by_id(student_id: int) -> Optional[Dict]:
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT id, full_name, roll_no, class, section, parent_id
        FROM students
        WHERE id = %s
    """,
        (student_id,),
    )
    row = cur.fetchone()
    cur.close()
    conn.close()
    if row:
        return {
```

```
            "id": row[0],
            "full_name": row[1],
            "roll_no": row[2],
            "class": row[3],
            "section": row[4],
            "parent_id": row[5],
        }
    return None


def get_students_by_class(cls: str, section: str = None) -> List[Dict]:
    conn = get_connection()
    cur = conn.cursor()
    if section:
        cur.execute(
            """
            SELECT id, full_name, roll_no, class, section
            FROM students
            WHERE class = %s AND section = %s
            ORDER BY roll_no
        """,
            (cls, section),
        )
    else:
        cur.execute(
            """
            SELECT id, full_name, roll_no, class, section
            FROM students
            WHERE class = %s
            ORDER BY roll_no
        """,
            (cls,),
        )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return [
        {
            "id": r[0],
            "full_name": r[1],
            "roll_no": r[2],
            "class": r[3],
            "section": r[4],
        }
        for r in rows
    ]
```

models/__init__.py

(blank)

models/schema.sql

```sql
CREATE DATABASE IF NOT EXISTS school_mgmt
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE school_mgmt;

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    email VARCHAR(100),
    is_active TINYINT(1) DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE roles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE user_roles (
    user_id INT NOT NULL,
    role_id INT NOT NULL,
    PRIMARY KEY (user_id, role_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
);
```

models/seed_roles.sql

```sql
INSERT INTO roles (name) VALUES
('admin'), ('principal'), ('teacher'), ('student'), ('parent'), ('office');
```

pages/0_DBA_Panels.py

```python
import streamlit as st
import mysql.connector
import bcrypt
```

```python
from auth.auth_manager import is_authenticated, has_role

st.set_page_config(page_title="DBA Panel", layout="centered")


def get_connection():
    return mysql.connector.connect(
        host=st.secrets["mysql"]["host"],
        port=st.secrets["mysql"]["port"],
        user=st.secrets["mysql"]["user"],
        password=st.secrets["mysql"]["password"],
        database=st.secrets["mysql"]["database"],
    )


def create_admin_user(username: str, password: str, full_name: str, email:
str):
    conn = get_connection()
    cur = conn.cursor()

    # hash password
    hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()

    # create user
    cur.execute(
        """
        INSERT INTO users (username, password_hash, full_name, email,
is_active)
        VALUES (%s, %s, %s, %s, 1)
        """,
        (username, hashed, full_name, email),
    )
    user_id = cur.lastrowid

    # assign admin role
    cur.execute(
        """
        INSERT INTO user_roles (user_id, role_id)
        VALUES (%s, (SELECT id FROM roles WHERE name='admin'))
        """,
        (user_id,),
    )

    conn.commit()
    cur.close()
    conn.close()
    return user_id
```

```
# ------------------ ACCESS CONTROL ------------------
if not is_authenticated() or not has_role("dba"):
    st.error("Access denied. Only Database Administrator (DBA) can access this
page.")
    st.stop()

# ------------------ UI ------------------
st.title("Database Administrator Panel")
st.caption("Only DBA can create school_app admin accounts.")

st.subheader("Create New Admin User")

with st.form("create_admin_form"):
    username = st.text_input("Admin Username")
    password = st.text_input("Admin Password", type="password")
    full_name = st.text_input("Full Name")
    email = st.text_input("Email")

    submitted = st.form_submit_button("Create Admin")

    if submitted:
        if not username or not password:
            st.error("Username and password are required.")
        else:
            try:
                user_id = create_admin_user(username, password, full_name,
email)
                st.success(
                    f"Admin user '{username}' created successfully (ID:
{user_id})."
                )
            except Exception as e:
                st.error(f"Error creating admin: {e}")
```

pages/1_Admin_Panle.py

```
import streamlit as st
import mysql.connector

from auth.auth_manager import is_authenticated, has_role

st.set_page_config(page_title="Admin Panel", layout="wide")


# ------------------ DB CONNECTION ------------------
```

```python
def get_connection():
    return mysql.connector.connect(
        host=st.secrets["mysql"]["host"],
        port=st.secrets["mysql"]["port"],
        user=st.secrets["mysql"]["user"],
        password=st.secrets["mysql"]["password"],
        database=st.secrets["mysql"]["database"],
    )


# ------------------ DB HELPERS ------------------
def get_all_users():
    conn = get_connection()
    cur = conn.cursor(dictionary=True)
    cur.execute(
        "SELECT id, username, full_name, email, is_active FROM users ORDER BY
id"
    )
    rows = cur.fetchall()
    cur.close()
    conn.close()
    return rows


def get_roles(user_id):
    conn = get_connection()
    cur = conn.cursor()
    cur.execute(
        """
        SELECT r.name
        FROM roles r
        JOIN user_roles ur ON ur.role_id = r.id
        WHERE ur.user_id = %s
    """,
        (user_id,),
    )
    roles = [r[0] for r in cur.fetchall()]
    cur.close()
    conn.close()
    return roles


def assign_role(user_id, role_name):
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        """
```

```
        INSERT INTO user_roles (user_id, role_id)
        VALUES (%s, (SELECT id FROM roles WHERE name=%s))
        ON DUPLICATE KEY UPDATE role_id = role_id
    """,
        (user_id, role_name),
    )

    conn.commit()
    cur.close()
    conn.close()


def remove_role(user_id, role_name):
    conn = get_connection()
    cur = conn.cursor()

    cur.execute(
        """
        DELETE FROM user_roles
        WHERE user_id=%s AND role_id=(SELECT id FROM roles WHERE name=%s)
    """,
        (user_id, role_name),
    )

    conn.commit()
    cur.close()
    conn.close()


def set_active_status(user_id, status):
    conn = get_connection()
    cur = conn.cursor()

    cur.execute("UPDATE users SET is_active=%s WHERE id=%s", (status,
user_id))

    conn.commit()
    cur.close()
    conn.close()


# ------------------ ACCESS CONTROL ------------------
if not is_authenticated() or not has_role("admin"):
    st.error("Access denied. Only Admin can access this page.")
    st.stop()


# ------------------ UI ------------------
```

```
st.title("Admin Panel")
st.caption("Manage user roles, permissions, and account status.")

users = get_all_users()

st.subheader("All Users")

for user in users:
    with st.expander(f"{user['username']} — {user['full_name']}"):
        st.write(f"**Email:** {user['email']}")
        st.write(f"**Active:** {'Yes' if user['is_active'] else 'No'}")

        current_roles = get_roles(user["id"])
        st.write(f"**Roles:** {', '.join(current_roles) if current_roles else
'None'}")

        st.markdown("---")

        # Assign new role
        st.write("### Assign Role")
        role_to_assign = st.selectbox(
            f"Assign role to {user['username']}",
            ["teacher", "office", "parent", "student"],
            key=f"assign_{user['id']}",
        )

        if st.button(f"Assign {role_to_assign}",
key=f"btn_assign_{user['id']}"):
            assign_role(user["id"], role_to_assign)
            st.success(f"Role '{role_to_assign}' assigned to
{user['username']}.")
            st.experimental_rerun()

        # Remove role
        st.write("### Remove Role")
        if current_roles:
            role_to_remove = st.selectbox(
                f"Remove role from {user['username']}",
                current_roles,
                key=f"remove_{user['id']}",
            )

            if st.button(f"Remove {role_to_remove}",
key=f"btn_remove_{user['id']}"):
                remove_role(user["id"], role_to_remove)
                st.warning(f"Role '{role_to_remove}' removed from
{user['username']}.")
```

```
                st.experimental_rerun()

        st.markdown("---")

        # Activate / Deactivate
        st.write("### Account Status")
        if user["is_active"]:
            if st.button("Deactivate Account", key=f"deact_{user['id']}"):
                set_active_status(user["id"], 0)
                st.warning(f"{user['username']} deactivated.")
                st.experimental_rerun()
        else:
            if st.button("Activate Account", key=f"act_{user['id']}"):
                set_active_status(user["id"], 1)
                st.success(f"{user['username']} activated.")
                st.experimental_rerun()
```

pages/1_Admin.py

```
import streamlit as st
from auth.auth_manager import is_authenticated, has_role

if not is_authenticated():
    st.error("Login required")
    st.stop()

if not has_role("admin"):
    st.error("Access denied")
    st.stop()

st.title("Admin Dashboard")
```

2_Teacher_Panle.py

```
import streamlit as st
from datetime import date

from auth.auth_manager import is_authenticated, has_role, logout
from db.students import get_students_by_class
from db.attendance import save_attendance
from db.marks import save_marks

st.set_page_config(page_title="Teacher Panel", layout="wide")

# ---------------------------------------------------------
# ACCESS CONTROL
# ---------------------------------------------------------
```

```python
if not is_authenticated():
    st.error("You must be logged in to view this page.")
    st.stop()

if not has_role("teacher", "admin"):
    st.error("You do not have permission to view this page.")
    st.stop()

teacher_id = st.session_state["user"]["id"]

# ---------------------------------------------------------
# HEADER
# ---------------------------------------------------------
st.title("Teacher Panel")
st.caption("Manage your classes, attendance, and student performance.")

col1, col2 = st.columns([2, 1])
with col1:
    st.subheader("My Classes")
    st.write("• Class 10-A – Mathematics")
    st.write("• Class 9-B – Science")

with col2:
    st.subheader("Quick Actions")
    st.button("Take Attendance")
    st.button("Enter Marks")
    st.button("Post Homework")

st.markdown("---")

# ---------------------------------------------------------
# ATTENDANCE SECTION
# ---------------------------------------------------------
st.subheader("Attendance")

selected_class = st.selectbox("Select Class", ["10", "9", "8", "7", "6"])
selected_section = st.selectbox("Section", ["A", "B", "C"])

students = get_students_by_class(selected_class, selected_section)

if not students:
    st.info("No students found for this class.")
    st.stop()

att_date = st.date_input("Date", value=date.today())

st.write(f"Attendance for Class {selected_class}-{selected_section}")
```

```
status_map = {}

for s in students:
    status = st.radio(
        f"{s['roll_no']} - {s['full_name']}",
        ["present", "absent"],
        horizontal=True,
        key=f"att_{s['id']}",
    )
    status_map[s["id"]] = status

if st.button("Save Attendance"):
    save_attendance(
        class_name=f"{selected_class}-{selected_section}",
        att_date=att_date,
        teacher_id=teacher_id,
        status_map=status_map,
    )
    st.success("Attendance saved successfully.")

st.markdown("---")

# ----------------------------------------------------------
# MARKS ENTRY SECTION
# ----------------------------------------------------------
st.subheader("Marks Entry")

exam_type = st.selectbox("Exam", ["Unit Test 1", "Unit Test 2", "Half Yearly",
"Final"])
subject = st.selectbox("Subject", ["Mathematics", "Science", "English"])

st.write(
    f"Enter marks for **Class {selected_class}-{selected_section}**,
**{subject}**, **{exam_type}**"
)

marks_entries = []

for s in students:
    m = st.number_input(
        f"{s['roll_no']} - {s['full_name']}",
        min_value=0,
        max_value=100,
        value=0,
        step=1,
        key=f"mark_{s['id']}",
```

```
    )

    marks_entries.append(
        {
            "student_id": s["id"],
            "subject": subject,
            "exam": exam_type,
            "marks_obtained": m,
            "max_marks": 100,
            "teacher_id": teacher_id,
        }
    )

if st.button("Save Marks"):
    save_marks(marks_entries)
    st.success("Marks saved successfully.")

st.markdown("---")

# ---------------------------------------------------------
# LOGOUT
# ---------------------------------------------------------
if st.button("Logout"):
    logout()
```

3_Student_Portal.py

```
# import streamlit as st
# from auth.auth_manager import is_authenticated, has_role, logout
from db.fees import get_fee_status, get_payment_history

with tab5:
    st.subheader("My Fees")

    student_id = user["student_id"]  # from login mapping

    status = get_fee_status(student_id)
    st.write(status)

    history = get_payment_history(student_id)
    st.write("Payment History:", history)
```

4_Parent_Portal.py

```
from db.fees import get_fee_status, get_payment_history
from db.parents import get_children_for_parent  # coming next

with tab4:
    st.subheader("Fees Status")

    student_id = selected_child_id  # from parent-child mapping

    status = get_fee_status(student_id)
    if status:
        st.write(f"Total Fees: ₹{status['total_fees']}")
        st.write(f"Paid: ₹{status['paid']}")
        st.write(f"Due: ₹{status['due']}")
        st.write(f"Last Payment: {status['last_payment_date']}")

        st.markdown("### Payment History")
        history = get_payment_history(student_id)
        for h in history:
            st.write(f"₹{h[0]} via {h[1]} on {h[2]}")
    else:
        st.info("No fee record found.")
```

5_Office_Desk.py

```
from db.fees import record_fee_payment, get_fee_status, get_payment_history
from db.students import get_student_by_roll  # we will create this next

# --- Fees Management ---
with tab2:
    st.subheader("Record Fee Payment")

    roll_no = st.text_input("Student Roll No")
    amount = st.number_input("Amount Paid", min_value=0, step=100)
    mode = st.selectbox("Payment Mode", ["Cash", "UPI", "Card", "Bank
Transfer"])

    if st.button("Record Payment"):
        student = get_student_by_roll(roll_no)
        if student:
            record_fee_payment(student["id"], amount, mode)
```

```
            st.success("Payment recorded successfully.")
        else:
            st.error("Student not found.")

    st.markdown("---")
    st.subheader("Check Fees Status")

    roll_search = st.text_input("Search Roll No", key="fees_search_roll")

    if st.button("Fetch Fees Status"):
        student = get_student_by_roll(roll_search)
        if student:
            status = get_fee_status(student["id"])
            st.write(status)
            history = get_payment_history(student["id"])
            st.write("Payment History:", history)
        else:
            st.error("Student not found.")
```

.gitignore

```
# -------------------------
# Python
# -------------------------
__pycache__/
*.pyc
*.pyo
*.pyd
*.pdb
*.pkl

# Virtual environments
venv/
env/
.venv/

# Jupyter
.ipynb_checkpoints/

# -------------------------
# Streamlit
# -------------------------
.streamlit/secrets.toml
.streamlit/*.json

# -------------------------
```

```
# Logs & temp files
# -------------------------
*.log
*.tmp
*.temp

# -------------------------
# OS-specific
# -------------------------
.DS_Store
Thumbs.db

# -------------------------
# IDE / Editor
# -------------------------
.vscode/
.idea/
*.code-workspace

# -------------------------
# MySQL dumps
# -------------------------
*.sql
*.sql.gz
*.dump

# -------------------------
# Cache
# -------------------------
.cache/
```

app.py

```
import streamlit as st
from auth.auth_manager import login_form, logout, is_authenticated, has_role

st.set_page_config(page_title="School Management System")

# if is_authenticated() and has_role("dba"):
#     st.sidebar.page_link("pages/0_DBA_Logout.py", label="DBA Logout")


def show_nav():
    st.sidebar.title("Navigation")
```

```
    if is_authenticated():
        st.sidebar.write(f"Welcome, {st.session_state['user']['full_name']}")

        if has_role("admin"):
            st.sidebar.page_link("pages/1_Admin.py", label="Admin Dashboard")
        if has_role("teacher"):
            st.sidebar.page_link("pages/2_Teacher.py", label="Teacher Panel")
        if has_role("student"):
            st.sidebar.page_link("pages/3_Student.py", label="Student Portal")

        if st.sidebar.button("Logout"):
            logout()
    else:
        st.sidebar.info("Please log in")


def main():
    if not is_authenticated():
        login_form()
        return

    show_nav()
    st.title("School Management System")


main()
```

README.md

```
# School Management System (Streamlit + Python + MySQL)

A modular, secure, and scalable **School Management System** built with:

- **Streamlit** (UI + routing)
- **Python** (backend logic)
- **MySQL 8 (WAMP)** (local database)
- **Role-Based Access Control (RBAC)** for Admin, Teacher, Student, Parent,
Office Staff

This project is designed to be clean, modular, and ready for future cloud
deployment.

---

## 🚀 Features
```

🔐 Authentication System
- Secure login with hashed passwords (bcrypt)
- Session-based authentication
- MySQL-backed user storage

🛡 Role-Based Access Control (RBAC)
- Admin
- Principal
- Teacher
- Student
- Parent
- Office Staff

Each role sees different pages and has different permissions.

🧩 Modular Architecture
```
school_app/
│
├── app.py
├── auth/
│   ├── auth_manager.py
│   ├── db_manager.py
│   └── init.py
├── pages/
│   ├── 1_Admin_Dashboard.py
│   ├── 2_Teacher_Panel.py
│   ├── 3_Student_Portal.py
│   ├── 4_Parent_Portal.py
│   └── 5_Office_Desk.py
└── .streamlit/
└── secrets.toml
```

🗄 Database Setup (MySQL 8 on WAMP)

Run this in phpMyAdmin:

```sql
CREATE DATABASE IF NOT EXISTS school_mgmt
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE school_mgmt;

CREATE TABLE users (
```

```
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(100),
    email VARCHAR(100),
    is_active TINYINT(1) DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE roles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE user_roles (
    user_id INT NOT NULL,
    role_id INT NOT NULL,
    PRIMARY KEY (user_id, role_id),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
);

INSERT INTO roles (name) VALUES
('admin'), ('principal'), ('teacher'), ('student'), ('parent'), ('office');
```

🔧 Configuration
Create:

Code
.streamlit/secrets.toml
Add:

toml
```
[mysql]
host = "localhost"
port = 3306
user = "root"
password = "YOUR_WAMP_PASSWORD"
database = "school_mgmt"
```
▶ Running the App
Install dependencies:

bash
```
pip install -r requirements.txt
```
Run Streamlit:

```
bash
streamlit run app.py
🌐 Future Cloud Deployment
This project is designed to migrate easily to:

AWS RDS

PlanetScale

Azure MySQL

Google Cloud SQL

Only the secrets.toml needs to change.

📄 License
MIT License

🧑‍💻 Author
Built by Jagdev Singh Dosanjh
Modular architect • Quant educator • Ed-tech builder

Code

---

# 🎁 Want me to generate the **full folder structure** with empty files?

I can produce:

- All folders
- All Python files with boilerplate
- Admin UI starter
- User creation script
- Role assignment script

Just say **"Generate full project skeleton"** and I'll build it.
```

requirements.txt

```
streamlit==1.32.0
mysql-connector-python==8.3.0
bcrypt==4.1.2
python-dotenv==1.0.1
```

```
pandas==2.2.1
matplotlib==3.8.3
plotly==5.19.0
openpyxl==3.1.2
```